

고객 정기 예금 가입 여부 예측 모델

2조 이름: 유재은 전예원 함예린



목차

1. 전처리 및 EDA
2. 모델링
3. 결론

전처리 및 EDA

라이브러리 импорт 및 데이터 불러오기

```
import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
```

#경고 메시지 숨기기

```
import warnings
warnings.filterwarnings(action='ignore')
```

```
from google.colab import drive
drive.mount('/content/drive')
file_path1 = '/content/drive/MyDrive/test_data.csv'
file_path2 = '/content/drive/MyDrive/train_data.csv'
test = pd.read_csv(file_path1)
train = pd.read_csv(file_path2)
```

결측치 처리

```
na_cols = ['CreditScore', 'AccountLengthYears', 'TenureWithBank', 'InvestmentPortfolioValue', 'CallResponseScore']

for col in na_cols:
    train[col] = train[col].fillna(train[col].median())
    test[col] = test[col].fillna(test[col].median())

cols_to_check = ['MarketingScore', 'ResponsePropensity', 'TermDepositSubscribed']
train = train.dropna(subset=cols_to_check)

fill_cols = ['MarketingScore', 'ResponsePropensity']
for c in fill_cols:
    test[c] = test[c].fillna(train[c].median())
```

- 'CreditScore', 'AccountLengthYears' 등의 결측치를 중앙값으로 채움
- train 데이터에서 'MarketingScore', 'ResponsePropensity', 'TermDepositSubscribed' 중 하나라도 결측치가 있는 행을 지움
- test 데이터에서 'MarketingScore', 'ResponsePropensity'의 결측치를 중앙값으로 채움

불필요한 피쳐 드롭

```
train = train.drop(columns=["ClientID", "TotalTransactions", "LastContactDay", "LastContactMonth"])
test = test.drop(columns=["ClientID", "TotalTransactions", "LastContactDay", "LastContactMonth"])
```

- ClientID: 분석에 불필요하므로 삭제
- TotalTransactions: NumOnlineTransactions과의 상관계수가 높는데 전체보다는 부분을 나타내는 NumOnlineTransactions가 예측에 더 용이하다고 판단하여 TotalTransactions는 삭제
- LastContactDay, LastContactMonth: 실제 예측 시점과 비교해서 언제가 가까운 날짜인지 비교할 수 없음 -> DaysSinceLastContact가 더 중요하다고 판단하여 삭제

‘Job Title’ , ‘Region’ 컬럼 처리

```
train['JobTitle'] = train['JobTitle'].str.lower().str.strip()
test['JobTitle'] = test['JobTitle'].str.lower().str.strip()
```

- ‘JobTitle’의 문자열들을 모두 소문자로 바꾸어줌

```
major_regions=["South","West","North","East","Metro","Central"]
train["Region"]=train["Region"].apply(
    lambda x: x if x in major_regions else "Other")
test["Region"]=test["Region"].apply(
    lambda x: x if x in major_regions else "Other")
```

- ‘Region’ 컬럼에서 ‘South’, ‘West’, ‘North’, ‘East’, ‘Metro’, ‘Central’ 이외의 값들은 그 수가 매우 적기 때문에 ‘Other’로 통칭함

JobGroup 파생변수 생성

```
def group_job(job):
    if job in ['management', 'admin', 'technician']:
        return 'high_stable'
    elif job in ['blue-collar', 'services', 'self-employed']:
        return 'mid_stable'
    elif job in ['retired', 'student', 'unemployed', 'entrepreneur']:
        return 'low_stable'
    else:
        return 'unknown'

# 새로운 파생 변수 생성
train['JobGroup'] = train['JobTitle'].apply(group_job)
test['JobGroup'] = test['JobTitle'].apply(group_job)
print(train.groupby('JobGroup')['TermDepositSubscribed'].mean().sort_values(ascending=False))
```

JobGroup	
high_stable	0.334495
mid_stable	0.278752
low_stable	0.278639

Name: TermDepositSubscribed, dtype: float64

직업별 안정도를 기준으로 직업을 3가지로 나눈 파생변수 JobGroup 생성

high_stable: management, admin, technician

mid_stable: blue-collar, services, self-employed

low_stable: retired, student, unemployed, entrepreneur

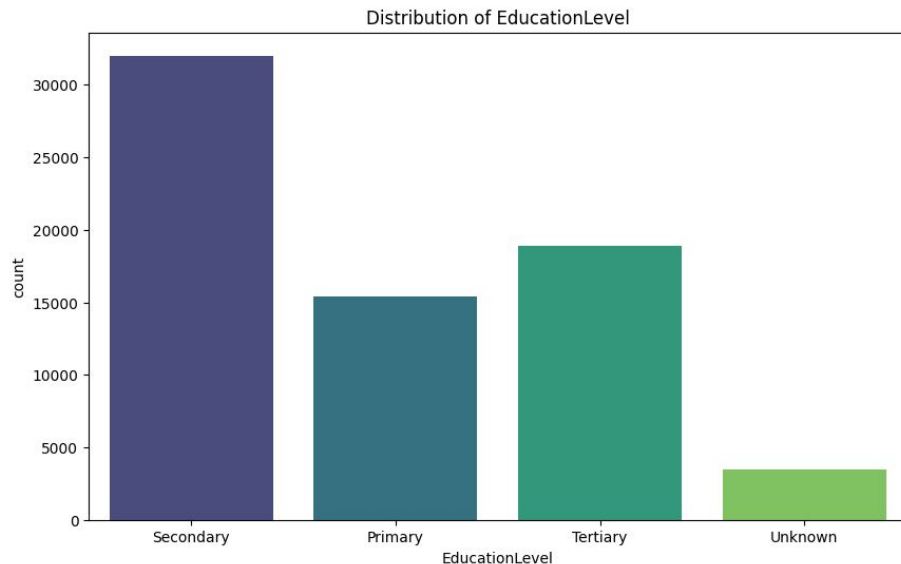
JobTitle 드롭

```
train = train.drop(columns=['JobTitle'], errors='ignore')  
test = test.drop(columns=['JobTitle'], errors='ignore')
```

- 파생변수 JobGroup 생성 후 필요없어진 JobTitle 제거

‘EducationLevel’ 컬럼 확인

```
plt.figure(figsize=(10, 6))
sns.countplot(x='EducationLevel', data=train, palette='viridis')
plt.title('Distribution of EducationLevel')
plt.show()
```



- ‘EducationLevel’을 count플롯으로 살펴본 결과 ‘Secondary’의 수가 가장 많고, 중간값에 해당하므로 추후 범주형 자료의 처리에서 ‘Unknown’을 ‘Secondary’과 같이 할당해줌

범주형변수 처리

```
# 1. Yes/No 형태의 이진 변수들을 0과 1로 변환
binary_cols = ['HasCreditCard', 'HasMortgage', 'HasPersonalLoan',
               'HasLifeInsurance', 'HasMutualFunds']

for col in binary_cols:
    train[col] = train[col].map({'Yes': 1, 'No': 0})
    test[col] = test[col].map({'Yes': 1, 'No': 0})

# 2. 순서가 있는 변수 (Ordinal Encoding)
edu_map = {'Primary': 0, 'Secondary': 1, 'Tertiary': 2, 'Unknown': 1}
train['EducationLevel'] = train['EducationLevel'].map(edu_map)
test['EducationLevel'] = test['EducationLevel'].map(edu_map)

sal_map = {'Low': 0, 'Mid': 1, 'UpperMid': 2, 'High': 3}
train['SalaryCategory'] = train['SalaryCategory'].map(sal_map)
test['SalaryCategory'] = test['SalaryCategory'].map(sal_map)

risk_map = {'Low': 0, 'Medium': 1, 'High': 2}
train['RiskRating'] = train['RiskRating'].map(risk_map)
test['RiskRating'] = test['RiskRating'].map(risk_map)
```

Yes/No 형태의 이진 변수: 0과 1로 변환
순서가 있는 변수:

- EducationLevel:
 - Primary:0, Secondary:1, Tertiary:2, Unknown:1
 - Unknown값은 가장 빈도가 높은 Secondary로 대체
- SalaryCategory:
 - Low:0, Mid:1, UpperMid:2, High:3
- RiskRating:
 - Low:0, Medium:1, High:2

범주형 변수 처리

```
# 3. 그 외 범주형 변수 (One-Hot Encoding)
ohe_cols = ['JobGroup', 'MaritalStatus', 'Gender', 'EmploymentStatus', 'Region', 'CustomerSegment', 'ChannelPreference', 'LastContactChannel', 'PrevCampaignOutcome']
train = pd.get_dummies(train, columns=ohe_cols, drop_first=True)
test = pd.get_dummies(test, columns=ohe_cols, drop_first=True)
```

pd.get_dummies 사용: 각 범주를 0/1이진칼럼으로 변환

drop_first=True: 기존 카테고리를 제거하여 다중공선성 방지

연속형 변수들의 로그 변환

```
log_cols = [  
    'AnnualIncome',  
    'NetWorth',  
    'CreditLimit',  
    'AccountBalance',  
    'InvestmentPortfolioValue',  
    'AvgTransactionValue',  
    'NumOnlineTransactions',  
    'NumMobileAppLogins',  
    'LastContactDuration'  
]  
  
for col in log_cols:  
    train[col] = np.log1p(train[col])  
    test[col] = np.log1p(test[col])
```

연속형 변수들의 로그 변환

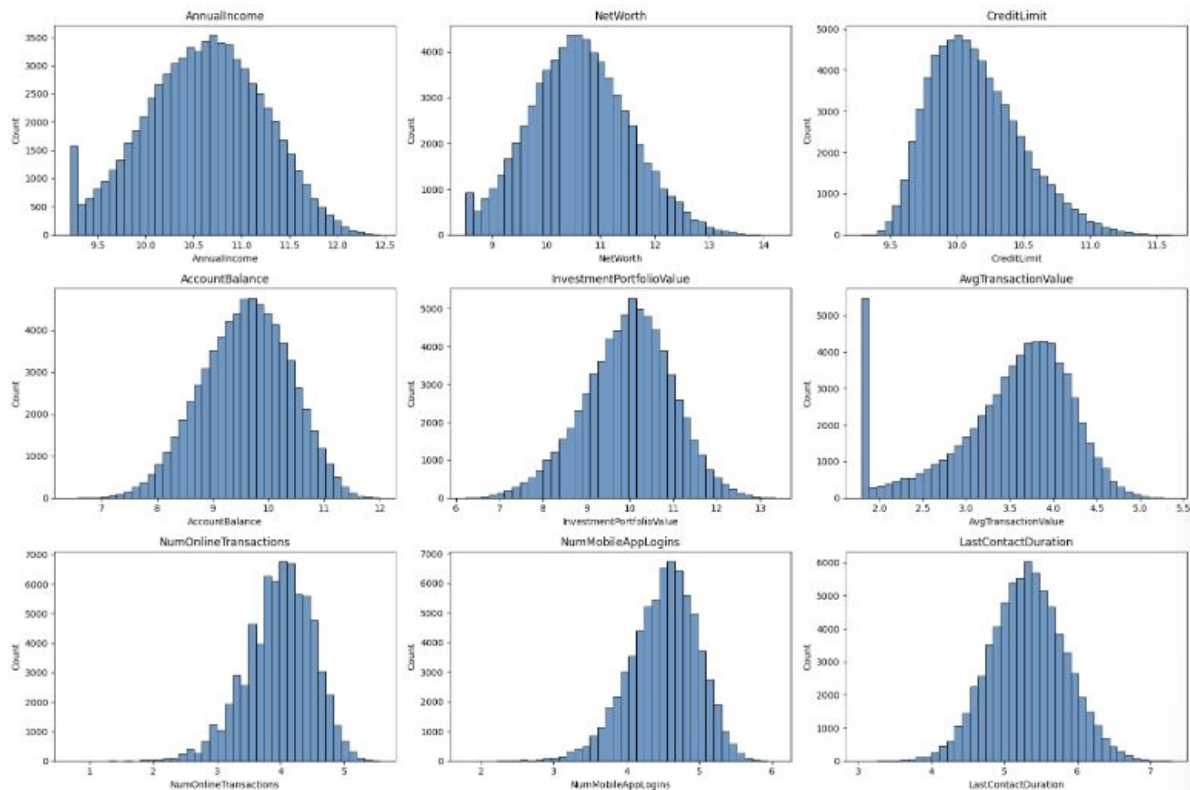
```
n_cols = 3
n_rows = int(np.ceil(len(log_cols) / n_cols))

plt.figure(figsize=(18, n_rows * 4))

for i, col in enumerate(log_cols, 1):
    plt.subplot(n_rows, n_cols, i)
    sns.histplot(train[col], bins=40)
    plt.title(col)

plt.tight_layout()
plt.show()
```

- 로그 변환 후, 정규분포에 가까워졌음을 확인할 수 있음



연속형 변수들의 로그 변환

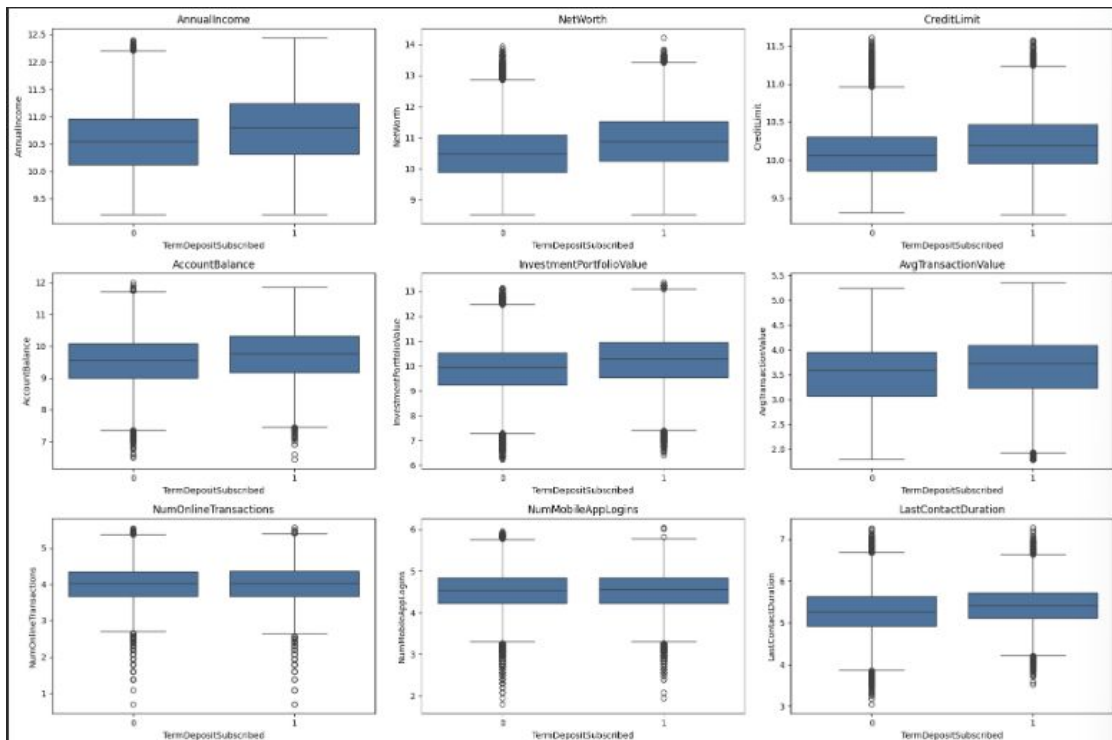
```
n_cols = 3
n_rows = int(np.ceil(len(log_cols) / n_cols))

plt.figure(figsize=(18, n_rows * 4))

for i, col in enumerate(log_cols, 1):
    plt.subplot(n_rows, n_cols, i)
    sns.boxplot(
        x='TermDepositSubscribed',
        y=col,
        data=train
    )
    plt.title(col)

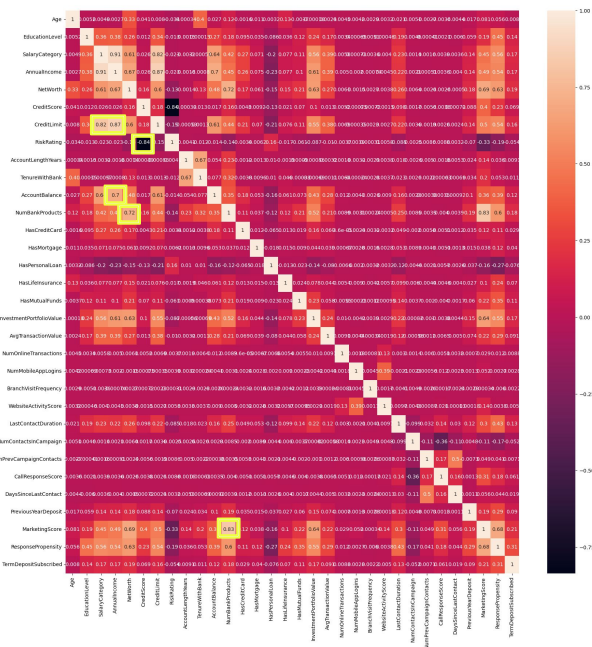
plt.tight_layout()
plt.show()
```

- 로그 변환 후, 이상치의 범위가 줄었음을 확인할 수 있음



변수 간의 상관관계 파악

```
plt.figure(figsize=(20,20))
sns.heatmap(data = train.select_dtypes(include=[np.number]).corr(), annot=True)
plt.show()
```



- 히트맵을 보면 일부 변수들 간의 상관관계가 상당히 높음
- 다중공선성을 제거하기 위해 상관계수가 높은 피쳐들 중 일부 피쳐를 제거해야 함

변수 간의 상관관계 파악

```
corr = train.select_dtypes(include=[np.number]).corr()
upper_tri = corr.abs().where(
    np.triu(np.ones(corr.shape), k=1).astype(bool)
)

high_corr = (
    upper_tri
    .stack()
    .reset_index()
    .rename(columns={
        "level_0": "Feature_1",
        "level_1": "Feature_2",
        0: "Correlation"
    })
)

high_corr = high_corr[high_corr["Correlation"] >= 0.7] #
    .sort_values("Correlation", ascending=False)

high_corr
```

	Feature_1	Feature_2	Correlation
61	SalaryCategory	AnnualIncome	0.908945
92	AnnualIncome	CreditLimit	0.867795
146	CreditScore	RiskRating	0.835658
303	NumBankProducts	MarketingScore	0.834651
64	SalaryCategory	CreditLimit	0.818257
124	NetWorth	NumBankProducts	0.717078

- 컬럼이 너무 많아서 히트맵에서 상관관계수가 높은 변수들을 한눈에 파악하기 어려움
- 상관관계수 절댓값이 0.7보다 높은 변수들만 추출함

상관관계가 높은 변수 제거

```
print(corr.loc["TermDepositSubscribed", "NumBankProducts"])  
print(corr.loc["TermDepositSubscribed", "MarketingScore"])  
print(corr.loc["TermDepositSubscribed", "NetWorth"])  
  
0.18148911947320662  
0.20555202532000322  
0.19015308631979064
```

```
train = train.drop(columns=["SalaryCategory", "CreditLimit", "NumBankProducts", "NetWorth"])  
test = test.drop(columns=["SalaryCategory", "CreditLimit", "NumBankProducts", "NetWorth"])
```

상관관계가 높게 나타났던 피처들과 타겟 피처의 상관계수 계산

그 중 타겟 피처와의 상관계수가 가장 높은 MarketingScore을 제외하고 다른 피처들은 drop

모델링

기초 성능 확인

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

target = "TermDepositSubscribed"

X = train.drop(columns=[target])
y = train[target].astype(int)

X, test_aligned = X.align(test, join="left", axis=1, fill_value=0)

X_tr, X_val, y_tr, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

pos = (y_tr == 1).sum()
neg = (y_tr == 0).sum()
pos_rate = pos / (pos + neg)
scale_pos_weight = neg / max(pos, 1)
```

기초 성능 확인

```
models = {  
    "LogisticRegression": Pipeline([  
        ("scaler", StandardScaler(with_mean=False)), # sparse/디미 대비  
        ("clf", LogisticRegression(  
            max_iter=5000,  
            class_weight="balanced",  
            solver="saga",  
            random_state=42  
        ))  
    ]),  
    "RandomForest": RandomForestClassifier(  
        class_weight="balanced",  
        random_state=42  
    ),  
    "GradientBoosting": GradientBoostingClassifier(  
        random_state=42  
    ),  
    "XGBoost": XGBClassifier(  
        scale_pos_weight=scale_pos_weight,  
        random_state=42  
    ),  
    "LightGBM": LGBMClassifier(  
        random_state=42,  
        scale_pos_weight=scale_pos_weight,  
        verbose=-1  
    )  
}
```

```
results = []  
  
for name, model in models.items():  
    model.fit(X_tr, y_tr)  
    pred_val = model.predict(X_val)  
    f1 = f1_score(y_val, pred_val)  
    results.append((name, f1))  
    print(f"{name:>18} | F1-Score = {f1:.4f}")  
  
results_df = pd.DataFrame(results, columns=["Model", "F1_valid"]).sort_values("F1_valid", ascending=False)  
print("\n==== F1(valid) Ranking ====")  
print(results_df)  
  
best_model_name = results_df.iloc[0]["Model"]  
best_model = models[best_model_name]  
print(f"\nBest model: {best_model_name}")
```

```
==== F1(valid) Ranking ====
```

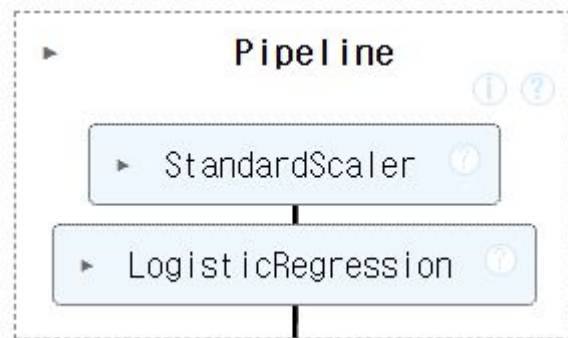
	Model	F1_valid
4	LightGBM	0.506800
0	LogisticRegression	0.505611
3	XGBoost	0.487680
2	GradientBoosting	0.316097
1	RandomForest	0.259232

Logistic Regression

-> LightGBM, XGBoost는 뚜렷한 성능 개선을 보이지 않는다는 것을 확인

```
Pipeline([  
    ("scaler", StandardScaler(with_mean=False)),  
    ("clf", LogisticRegression(  
        max_iter=5000,  
        class_weight="balanced",  
        solver="saga",  
        random_state=42  
    ))  
])
```

```
logit_model = models["LogisticRegression"]  
logit_model.fit(X, y)
```



Logistic Regression

- 피쳐 중요도 확인

```
import numpy as np
import pandas as pd

coef = logit_model.named_steps["clf"].coef_[0]
features = X.columns

logit_importance = pd.DataFrame({
    "Feature": features,
    "Coefficient": coef,
    "AbsCoefficient": np.abs(coef)
}).sort_values("AbsCoefficient", ascending=False)

logit_importance.head(15)
logit_importance.tail(15)
```

	Feature	Coefficient	AbsCoefficient
12	HasMutualFunds	-0.004401	0.004401
35	EmploymentStatus_Student	-0.004353	0.004353
14	AvgTransactionValue	0.004262	0.004262
50	PrevCampaignOutcome_Success	0.003266	0.003266
39	Region_North	-0.002800	0.002800
2	AnnualIncome	0.002779	0.002779
34	EmploymentStatus_Self-employed	-0.002358	0.002358
37	Region_East	-0.002227	0.002227
20	NumContactsInCampaign	-0.002070	0.002070
24	PreviousYearDeposit	0.001987	0.001987
33	EmploymentStatus_Retired	0.001259	0.001259
5	AccountLengthYears	-0.000994	0.000994
6	TenureWithBank	-0.000448	0.000448
16	NumMobileAppLogins	-0.000411	0.000411
11	HasLifeInsurance	0.000130	0.000130

Logistic Regression

- 영향력이 낮은 변수 제거: 제거했을 때 성능이 개선되는 변수만!

```
drop_cols_1 = [  
    "NumMobileAppLogins",  
    "AccountLengthYears",  
]  
  
X_reduced = X.drop(columns=drop_cols_1)  
  
X_tr2, X_val2, y_tr2, y_val2 = train_test_split(  
    X_reduced, y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)  
  
logit_model.fit(X_tr2, y_tr2)  
pred = logit_model.predict(X_val2)  
  
print("F1 after removal:", f1_score(y_val2, pred))
```

약 **0.5056** -> 약 **0.5063**
으로
소폭 상승

Logistic Regression

- threshold 조정

```
import numpy as np
from sklearn.metrics import f1_score

logit_model.fit(X_tr2, y_tr2)

val_proba = logit_model.predict_proba(X_val2)[:, 1]

thresholds = np.linspace(0.05, 0.95, 181)
f1s = []

for t in thresholds:
    preds = (val_proba >= t).astype(int)
    f1s.append(f1_score(y_val2, preds))

best_idx = int(np.argmax(f1s))
best_thr = thresholds[best_idx]
best_f1 = f1s[best_idx]

print(f"Best threshold = {best_thr:.3f}")
print(f"Best F1-Score = {best_f1:.4f}")
```

Best threshold = 0.430
Best F1-Score = 0.5160

Logistic Regression

```
test_reduced = test_aligned.drop(columns=drop_cols_1, errors="ignore")
test_reduced = test_reduced.reindex(columns=X_reduced.columns, fill_value=0)

logit_model = models["LogisticRegression"]
logit_model.fit(X_reduced, y)

test_proba = logit_model.predict_proba(test_reduced)[: , 1]
test_pred = (test_proba >= best_thr).astype(int)

print("sub rows:", len(sub))
print("pred len:", len(test_pred))
print("test rows:", len(test_reduced))

sub = pd.read_csv("sample_submission.csv")
sub["TermDepositSubscribed"] = test_pred
sub.to_csv("submission_logit1.csv", index=False)
sub.head()
```

최종 제출 점수: 0.51457403

결론

결론 및 느낀점

- 최종 선정 모델: Logistic Regression
- 최종 성능: 0.51457403

< 느낀점 >

- 앙상블을 적용한 것보다 단일 모델이 더 좋은 성능을 보일 수 있음
- threshold(임계값)을 조정하는 것이 성능 개선에 도움이 됨

감사합니다